DATABASE MANAGEMENT SYSTEMS · PROJECT · PHASE 3

# MUSE — Music Mood Intelligence & Recommendation Database System

Querying, Interface and Project Report

| | |
|---|---|
| Submitted by | Vaishnav Sai |
| Roll number | |
| Programme | B.Tech CSE |
| Course | Database Management Systems |
| Submitted to | |
| Date | |

MySQL 8.0.46 Community Server · MySQL Workbench 8.0 CE · Streamlit interface. All outputs in this report were produced by running the queries on the Phase 2 database.

## 1. Project summary

**MUSE** is a database for a music service that understands *mood*. Every song has measured audio features (energy, valence/positivity, danceability, acousticness, tempo). A versioned mood model turns these into mood tags with a confidence, listeners' plays, skips, ratings and playlists are recorded, and the system builds explainable recommendations and a weekly personalised "MUSE World" from them. The catalogue is Indian multilingual music (12 languages), so the design also handles scripts, regions and film soundtracks.

Phase 1 designed the ER model and normalised it; Phase 2 created the schema (DDL) and loaded sample data (DML). This phase adds 17 queries that answer real questions about the data, an interface that runs them and shows the results as tables and charts, and this report.

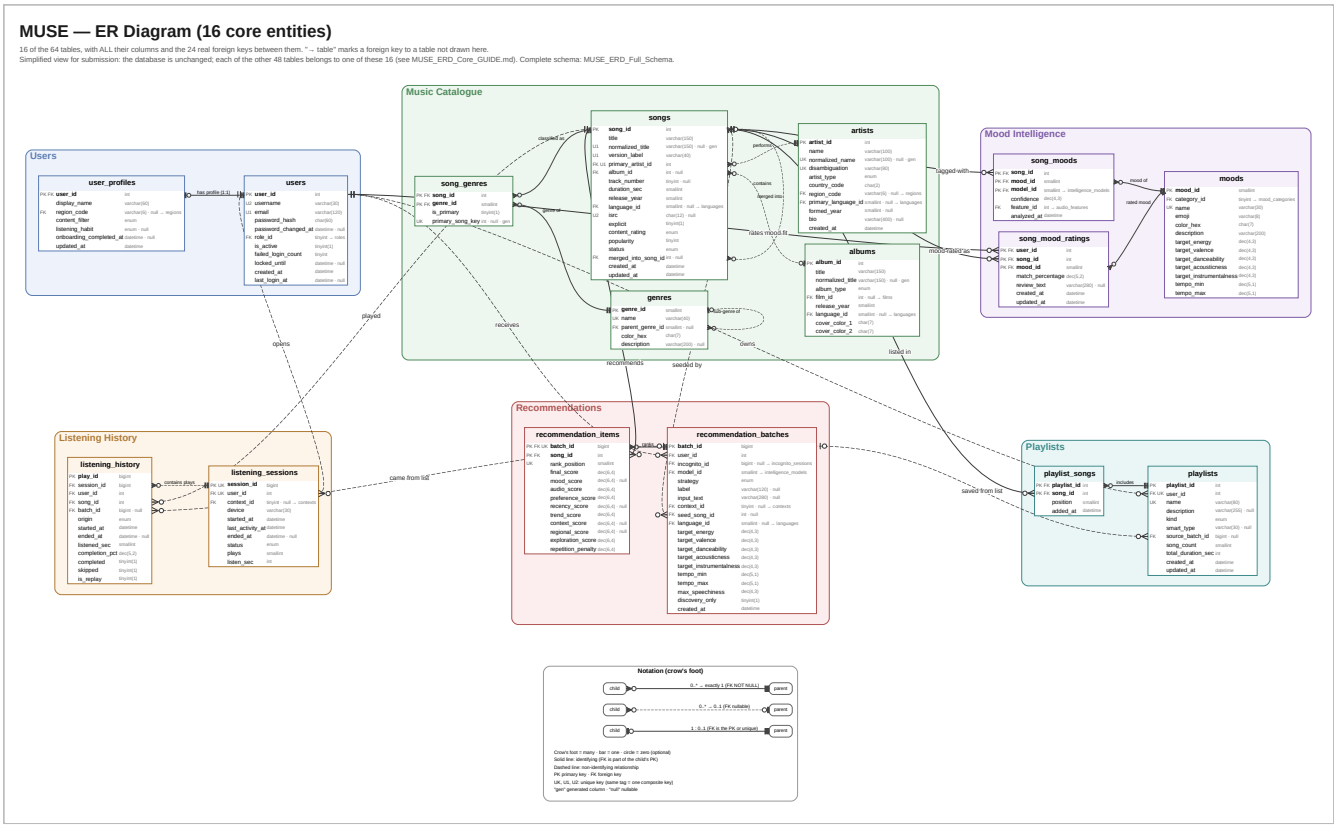
| Phase 3 deliverable | File |
|---|---|
| SQL query file | `database/phase3/MUSE_Phase3_Queries.sql` — 17 queries with headers; runs in Workbench (Ctrl + Enter per query) |
| Screenshots of query + output | `docs/phase3/screenshots/` — one per query plus the dashboard, mood explorer and SQL playground (21 PNG files) |
| Project report (PDF) | this document |
| Frontend files | `phase3/app/` — `streamlit_app.py` (MUSE Query Studio), `muse_cli.py` (command line), `muse_queries.py`, `requirements.txt`, `.env.example` |

## 2. Schema

The database `muse_db` has 64 tables in six areas, with 64 primary keys, 124 foreign keys, 39 unique keys and 71 CHECK constraints (full catalogue in the Phase 2 report).

| Area | Tables | Rows | Contents |
|---|---|---|---|
| Users | 11 | 110 | accounts, roles, permissions, profiles, languages, regions, sign-in sessions, audit log |
| Music Catalogue | 18 | 3,232 | artists, albums, films, songs, genres, music sources, import pipeline |
| Mood Intelligence | 15 | 1,910 | moods, contexts, audio features, AI mood tags with model versions, user mood-match ratings |
| Listening History | 9 | 9,289 | sessions, plays, playback events, ratings, likes, dismissals, searches |
| Recommendations | 9 | 1,108 | recommendation batches, ranked songs with reasons, weekly MUSE Worlds |
| Playlists | 2 | 63 | user playlists and their songs |
| **Total** | **64** | **15,712** | |

The core entities and their relationships (16 central tables; generated from the live schema):


MUSE — ER Diagram (16 core entities)
16 of the 64 tables, with ALL their columns and the 24 real foreign keys between them. " – table" marks a foreign key to a table not drawn here.
Simplified view for submission: the database is unchanged; each of the other 48 tables belongs to one of these 16 (see MUSE_ERD_Core_GUIDE.md). Complete schema: MUSE_ERD_Full_Schema.
Users
user_profiles
users
Music Catalogue
songs
song_genres
genres
artists
albums
Mood Intelligence
song_moods
song_mood_ratings
moods
Listening History
listening_history
listening_sessions
Recommendations
recommendation_items
recommendation_batches
Playlists
playlist_songs
playlists
Notation (crow's foot)
child
parent
0..* – ... exactly 1 (FK NOT NULL)
0..* – ... 0..1 (FK nullable)
1 .. 0..1 (FK is the PK or unique)

## 3. Sample data

15,712 rows are loaded by `schema.sql`: 13 users (one admin, one curator and 11 listeners), 104 songs by 31 artists on 32 albums, 726 mood tags, 497 listening sessions with 2,634 plays and 5,942 playback events, 146 star ratings, 8 playlists and 198 recommended songs with 770 reasons. The data is demo data made for the project (no real people).

| Table | Rows | Table | Rows |
|---|---|---|---|
| users | 13 | artists | 31 |
| albums | 32 | songs | 104 |
| moods | 17 | song_moods | 726 |
| audio_features | 208 | listening_sessions | 497 |
| listening_history | 2,634 | ratings | 146 |
| playlists | 8 | playlist_songs | 55 |
| recommendation_items | 198 | weekly_worlds | 11 |

Largest tables: playback_events (5,942), listening_history (2,634), trend_snapshots (2,109), recommendation_reasons (770), song_moods (726).

## 4. Queries and outputs

All queries are in `MUSE_Phase3_Queries.sql`. Each has a header (`-- @id`, `@title`, `@concepts`, `@question`) that the interface reads, so the file is the single source for Workbench, the Streamlit app and the CLI. Below, each query is shown with its first rows of real output; the screenshots show the complete output.

### 4.1 Coverage of the required concepts

● = used in that query. Every required concept is covered by at least one query; the last three rows are additional techniques.

| Concept | 01 | 02 | 03 | 04 | 05 | 06 | 07 | 08 | 09 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | # |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| SELECT | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | **17** |
| WHERE | ● | ● | | | | | | | ● | | | | ● | | | ● | | **5** |
| SUM | | | ● | ● | | ● | | | | | ● | | ● | ● | ● | ● | | **8** |
| AVG | | | ● | ● | ● | ● | ● | | | ● | ● | ● | ● | | ● | | ● | **11** |
| COUNT | | | ● | ● | ● | ● | ● | ● | | ● | ● | ● | ● | ● | | ● | ● | **13** |
| MIN | | | ● | | ● | | | | | | | ● | | | | | | **3** |
| MAX | | | ● | ● | ● | | | | | | | ● | | | | | ● | **5** |
| GROUP BY | | | | ● | ● | ● | ● | ● | | ● | ● | ● | ● | ● | ● | ● | ● | **13** |
| HAVING | | | | | ● | ● | | | | | | ● | | | ● | | | **4** |
| ORDER BY | ● | ● | | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | ● | **16** |
| INNER JOIN | | ● | | ● | ● | ● | ● | ● | ● | | | ● | | ● | ● | ● | ● | **12** |
| LEFT JOIN | | | | | | | | ● | ● | | ● | | ● | ● | | | | **5** |
| RIGHT JOIN | | | | | | | | | | ● | | | | | | | | **1** |
| Subquery | | | | | | | | | | | | | ● | | ● | ● | | **3** |
| CTE (WITH) | | | | | | | | | | | | | ● | | | ● | | **2** |
| Window function | | | | | | | | | | | | | | | | ● | | **1** |

## Q01 · Popular recent songs

*Which active songs released in 2022 or later have a popularity of at least 75?*

SELECT | WHERE (= | >= | AND) | ORDER BY | LIMIT

```
SELECT song_id, title, release_year, popularity, duration_sec
FROM songs
WHERE status = 'ACTIVE'
  AND release_year >= 2022
  AND popularity >= 75
ORDER BY popularity DESC, title
LIMIT 15;
```

Output: 15 rows in 0.9 ms · screenshot Q01_popular_recent_songs.png

| song_id | title | release_year | popularity | duration_sec |
|---|---|---|---|---|
| 1 | Vennela Vaana | 2022 | 92 | 228 |
| 47 | Safar-e-Ishq | 2022 | 91 | 278 |
| 31 | Kuthu Beat | 2024 | 90 | 222 |
| 15 | Yuddham | 2023 | 88 | 231 |
| 34 | Raat Ki Rani | 2023 | 87 | 213 |
| 44 | Dhamaal | 2024 | 86 | 222 |
| 17 | Gelupu Gelupu | 2023 | 86 | 242 |
| 75 | Nachdi Raat | 2023 | 85 | 244 |

… 7 more rows (all rows in the screenshot and the app)

## Q02 · Calm-sounding tracks by audio profile

*Which songs sound calm — low energy, slow tempo and mostly acoustic — according to their current audio analysis?*

SELECT | WHERE (BETWEEN) | INNER JOIN | ORDER BY

```
SELECT s.title, a.name AS artist,
       f.energy, f.valence, f.acousticness, f.tempo_bpm
FROM audio_features f
INNER JOIN songs   s ON s.song_id   = f.song_id
INNER JOIN artists a ON a.artist_id = s.primary_artist_id
WHERE f.is_current = 1
  AND f.energy BETWEEN 0 AND 0.30
  AND f.tempo_bpm BETWEEN 60 AND 105
  AND f.acousticness >= 0.40
ORDER BY f.energy, f.tempo_bpm;
```

Output: 19 rows in 0.9 ms · screenshot Q02_calm_sounding_tracks_by_audio_profile.png

| title | artist | energy | valence | acousticness | tempo_bpm |
|---|---|---|---|---|---|
| Snowfield Breathing | Aster Fields | 0.096 | 0.525 | 0.77 | 62.1 |
| Glacier Hymn I | Aster Fields | 0.097 | 0.484 | 0.794 | 61.3 |
| Majuli Mist | Brahmaputra Drift | 0.105 | 0.442 | 0.837 | 74 |
| Northern Lull | Aster Fields | 0.112 | 0.481 | 0.836 | 68.1 |
| Mahanadi Lullaby | Konark Skyline | 0.159 | 0.538 | 0.761 | 63.5 |
| Lantern Waltz | Elena Varga | 0.166 | 0.41 | 0.935 | 66.1 |
| Thillana in Blue | Carnatic Circuit | 0.187 | 0.406 | 0.984 | 64 |
| Nocturne for an Empty Room | Elena Varga | 0.202 | 0.405 | 0.939 | 63.7 |

… 11 more rows (all rows in the screenshot and the app)

## Q03 · Platform listening summary

*How much listening does the platform hold in total, and over what period?*

COUNT · COUNT DISTINCT · SUM · AVG · MIN · MAX

```
SELECT COUNT(*)                          AS total_plays,
       COUNT(DISTINCT user_id)           AS listeners,
       COUNT(DISTINCT song_id)           AS distinct_songs,
       ROUND(SUM(listened_sec) / 3600, 1) AS hours_listened,
       ROUND(AVG(completion_pct), 1)     AS avg_completion_pct,
       SUM(completed)                    AS completed_plays,
       SUM(skipped)                      AS skipped_plays,
       MIN(started_at)                   AS first_play,
       MAX(started_at)                   AS last_play
FROM listening_history;
```

Output: 1 row in 2.1 ms · screenshot Q03_platform_listening_summary.png

| total_plays | listeners | distinct_songs | hours_listened | avg_completion_pct | completed_plays | skipped_plays | first_play | last_play |
|---|---|---|---|---|---|---|---|---|
| 2634 | 11 | 104 | 142.4 | 82.3 | 1936 | 374 | 2026-08-28 01:19:00 | 2026-10-26 14:58:15 |

## Q04 · Listener leaderboard

*Who listens the most, how completely do they listen, and when did they last play something?*

INNER JOIN · GROUP BY · COUNT · SUM · AVG · MAX · ORDER BY

```
SELECT p.display_name                        AS listener,
       u.username,
       COUNT(*)                              AS plays,
       ROUND(SUM(lh.listened_sec) / 3600, 1)   AS hours,
       ROUND(AVG(lh.completion_pct), 1)        AS avg_completion_pct,
       ROUND(100 * AVG(lh.skipped), 1)         AS skip_rate_pct,
       MAX(lh.started_at)                    AS last_played
FROM listening_history lh
INNER JOIN users         u ON u.user_id = lh.user_id
INNER JOIN user_profiles p ON p.user_id = u.user_id
GROUP BY u.user_id, p.display_name, u.username
ORDER BY hours DESC;
```

Output: 11 rows in 5.7 ms · screenshot Q04_listener_leaderboard.png

| listener | username | plays | hours | avg_completion_pct | skip_rate_pct | last_played |
|---|---|---|---|---|---|---|
| Rohan Kulkarni | rohan | 270 | 15.3 | 80.6 | 14.8 | 2026-10-26 12:54:19 |
| Meera Iyer | meera | 276 | 15.2 | 85.2 | 11.2 | 2026-10-26 14:57:16 |
| Aarav Varma | aarav | 289 | 14.2 | 80.1 | 17.3 | 2026-10-26 14:07:04 |
| Ishita Sen | ishita | 250 | 13.8 | 81.8 | 14 | 2026-10-26 13:59:45 |
| Priya Naidu | priya | 262 | 13.7 | 81.4 | 15.3 | 2026-10-26 14:45:11 |
| Arjun Shetty | arjun | 233 | 13.5 | 85.9 | 9.9 | 2026-10-26 13:38:22 |
| Sana Qureshi | sana | 260 | 13.1 | 81.5 | 15.8 | 2026-10-26 14:58:15 |
| Nilakshi Bora | nila | 199 | 12.9 | 81.6 | 15.6 | 2026-10-26 12:56:02 |

… 3 more rows (all rows in the screenshot and the app)

## Q05 · Artists with a strong catalogue

*Which artists have at least 4 songs and an average popularity above 70?*

INNER JOIN · GROUP BY · HAVING · COUNT · AVG · MIN · MAX · ORDER BY

```
SELECT a.name                     AS artist,
       COUNT(*)                   AS songs,
       ROUND(AVG(s.popularity), 1)  AS avg_popularity,
       MIN(s.release_year)        AS first_release,
       MAX(s.release_year)        AS latest_release
FROM artists a
INNER JOIN songs s ON s.primary_artist_id = a.artist_id
GROUP BY a.artist_id, a.name
HAVING COUNT(*) >= 4 AND AVG(s.popularity) > 70
ORDER BY avg_popularity DESC;
```

Output: 8 rows in 0.8 ms · screenshot Q05_artists_with_a_strong_catalogue.png

| artist | songs | avg_popularity | first_release | latest_release |
|---|---|---|---|---|
| Ishaan Kheda | 5 | 82.6 | 2022 | 2025 |
| Rama Chandrasekhar | 4 | 81.5 | 2023 | 2023 |
| Harish Velmurugan | 4 | 81 | 2024 | 2024 |
| Kabir Rawat | 4 | 80.3 | 2023 | 2023 |
| Varsha Kondapalli | 6 | 78.5 | 2022 | 2024 |
| Ilan Mathivanan | 4 | 77.5 | 2021 | 2024 |
| Mumbai Lo-Fi Society | 4 | 74.5 | 2024 | 2025 |
| Godavari Echoes | 4 | 73.3 | 2021 | 2025 |

## Q06 · Most-skipped songs

*Among songs played at least 20 times, which are skipped most often?*

INNER JOIN · GROUP BY · HAVING · COUNT · SUM · AVG · ORDER BY

```
SELECT s.title,
       a.name                              AS artist,
       COUNT(*)                            AS plays,
       SUM(lh.skipped)                     AS skips,
       ROUND(100 * SUM(lh.skipped) / COUNT(*), 1) AS skip_rate_pct,
       ROUND(AVG(lh.listened_sec))         AS avg_seconds_listened
FROM listening_history lh
INNER JOIN songs   s ON s.song_id   = lh.song_id
INNER JOIN artists a ON a.artist_id = s.primary_artist_id
GROUP BY s.song_id, s.title, a.name
HAVING COUNT(*) >= 20
ORDER BY skip_rate_pct DESC, plays DESC
LIMIT 10;
```

Output: 10 rows in 5.3 ms · screenshot Q06_most_skipped_songs.png

| title | artist | plays | skips | skip_rate_pct | avg_seconds_listened |
|---|---|---|---|---|---|
| Nandi Hills Run | Ranga Collective | 26 | 8 | 30.8 | 163 |
| Majuli Mist | Brahmaputra Drift | 24 | 7 | 29.2 | 193 |
| Charminar Nights | Kaali Beats | 29 | 8 | 27.6 | 143 |
| Library After Hours | The Quiet Arcade | 26 | 7 | 26.9 | 106 |
| Sahyadri Sunrise | Sahyadri Strings | 35 | 9 | 25.7 | 205 |
| Chinukulu | Varsha Kondapalli | 20 | 5 | 25 | 206 |
| Safar-e-Ishq | Ishaan Kheda | 29 | 7 | 24.1 | 198 |
| Monsoon Mornings in Kochi | Backwater Radio | 23 | 5 | 21.7 | 241 |

… 2 more rows (all rows in the screenshot and the app)

## Q07 · Mood map of the catalogue

*For each mood, how many songs does the active mood model tag with it, how confident is it, and what do those songs sound like on average?*

INNER JOIN (4 tables) GROUP BY COUNT AVG ORDER BY

```
SELECT m.name                          AS mood,
       COUNT(*)                        AS songs,
       ROUND(AVG(sm.confidence), 3)    AS avg_confidence,
       ROUND(AVG(f.energy), 2)         AS avg_energy,
       ROUND(AVG(f.valence), 2)        AS avg_valence,
       ROUND(AVG(f.tempo_bpm))         AS avg_tempo_bpm
FROM song_moods sm
INNER JOIN moods              m  ON m.mood_id   = sm.mood_id
INNER JOIN intelligence_models im ON im.model_id = sm.model_id AND im.is_active = 1
INNER JOIN audio_features      f  ON f.feature_id = sm.feature_id
GROUP BY m.mood_id, m.name
ORDER BY songs DESC;
```

Output: 17 rows in 1.9 ms · screenshot Q07_mood_map_of_the_catalogue.png

| mood | songs | avg_confidence | avg_energy | avg_valence | avg_tempo_bpm |
|---|---|---|---|---|---|
| Nostalgic | 45 | 0.747 | 0.49 | 0.55 | 97 |
| Dreamy | 40 | 0.695 | 0.49 | 0.54 | 98 |
| Relaxed | 36 | 0.761 | 0.39 | 0.61 | 90 |
| Romantic | 35 | 0.824 | 0.44 | 0.6 | 95 |
| Motivational | 33 | 0.732 | 0.83 | 0.71 | 125 |
| Calm | 31 | 0.686 | 0.27 | 0.49 | 77 |
| Focused | 26 | 0.738 | 0.32 | 0.51 | 82 |
| Euphoric | 24 | 0.776 | 0.86 | 0.78 | 128 |

… 9 more rows (all rows in the screenshot and the app)

## Q08 · Every mood with its song count

*How many songs does each of the 17 moods have? A mood with no songs would still be listed, with 0 (an INNER JOIN would drop it).*

LEFT JOIN GROUP BY COUNT ORDER BY

```
SELECT mc.name                 AS mood_family,
       m.name                  AS mood,
       COUNT(sm.song_id)       AS songs_tagged
FROM moods m
INNER JOIN mood_categories mc ON mc.category_id = m.category_id
LEFT JOIN (song_moods sm
           INNER JOIN intelligence_models im ON im.model_id = sm.model_id AND im.is_active = 1)
       ON sm.mood_id = m.mood_id
GROUP BY mc.name, m.mood_id, m.name
ORDER BY mood_family, songs_tagged DESC;
```

Output: 17 rows in 1.2 ms · screenshot Q08_every_mood_with_its_song_count.png

| mood_family | mood | songs_tagged |
|---|---|---|
| Bright & lively | Motivational | 33 |
| Bright & lively | Euphoric | 24 |
| Bright & lively | Happy | 24 |
| Bright & lively | Confident | 23 |
| Bright & lively | Energetic | 21 |
| Bright & lively | Adventurous | 13 |
| Intense | Angry | 2 |
| Reflective | Nostalgic | 45 |

… 9 more rows (all rows in the screenshot and the app)

## Q09 · Songs no one has put in a playlist

*Which popular songs have never been added to any playlist?*

LEFT JOIN … IS NULL (anti-join) | WHERE | ORDER BY

```
SELECT s.title, a.name AS artist, s.popularity
FROM songs s
INNER JOIN artists a ON a.artist_id = s.primary_artist_id
LEFT JOIN playlist_songs ps ON ps.song_id = s.song_id
WHERE ps.song_id IS NULL
  AND s.popularity >= 70
ORDER BY s.popularity DESC;
```

Output: 28 rows in 0.8 ms · screenshot Q09_songs_no_one_has_put_in_a_playlist.png

| title | artist | popularity |
|---|---|---|
| Safar-e-Ishq | Ishaan Kheda | 91 |
| Yuddham | Rama Chandrasekhar | 88 |
| Raat Ki Rani | Kabir Rawat | 87 |
| Baarish Ki Chitthi | Saffron Skies | 85 |
| Gunde Chappudu | Varsha Kondapalli | 84 |
| Dil Ki Dhoop | Kabir Rawat | 83 |
| Ponnu Paarvai | Harish Velmurugan | 82 |
| Hyderabadi Hustle | Kaali Beats | 81 |

… 20 more rows (all rows in the screenshot and the app)

## Q10 · Catalogue by language

*How many songs exist in each language? Every language must be listed, even one with no songs.*

RIGHT JOIN | GROUP BY | COUNT | AVG | ORDER BY

```
SELECT l.name                     AS language,
       l.script,
       COUNT(s.song_id)           AS songs,
       ROUND(AVG(s.popularity), 1)  AS avg_popularity
FROM songs s
RIGHT JOIN languages l ON l.language_id = s.language_id
GROUP BY l.language_id, l.name, l.script
ORDER BY songs DESC, language;
```

Output: 12 rows in 0.8 ms · screenshot Q10_catalogue_by_language.png

| language | script | songs | avg_popularity |
|---|---|---|---|
| Hindi | Devanagari | 18 | 78.9 |
| Telugu | Telugu | 16 | 77.6 |
| Tamil | Tamil | 11 | 75.7 |
| Bengali | Bengali | 6 | 67 |
| Kannada | Kannada | 6 | 70.7 |
| Punjabi | Gurmukhi | 6 | 76.8 |
| Assamese | Bengali-Assamese | 3 | 60.7 |
| Malayalam | Malayalam | 3 | 73.3 |

… 4 more rows (all rows in the screenshot and the app)

## Q11 · Listening by context

*In which situations (study, workout, travel …) do people listen most, and how completely?*

LEFT JOIN · GROUP BY · COUNT · SUM · AVG · ORDER BY

```
SELECT COALESCE(c.name, 'No context')       AS context,
       COUNT(DISTINCT ls.session_id)         AS sessions,
       COUNT(lh.play_id)                     AS plays,
       ROUND(SUM(lh.listened_sec) / 3600, 1) AS hours,
       ROUND(AVG(lh.completion_pct), 1)      AS avg_completion_pct
FROM listening_sessions ls
LEFT JOIN contexts c           ON c.context_id = ls.context_id
LEFT JOIN listening_history lh ON lh.session_id = ls.session_id
GROUP BY c.context_id, c.name
ORDER BY hours DESC;
```

Output: 11 rows in 4.9 ms · screenshot Q11_listening_by_context.png

| context | sessions | plays | hours | avg_completion_pct |
|---|---|---|---|---|
| No context | 242 | 1268 | 68.3 | 82.2 |
| Late night | 44 | 243 | 13.7 | 84 |
| Workout | 43 | 223 | 11.6 | 82.8 |
| Evening | 39 | 213 | 11.6 | 81.1 |
| Party | 29 | 168 | 8.4 | 81.9 |
| Morning | 25 | 138 | 7.8 | 81.5 |
| Travel | 18 | 95 | 5.2 | 80 |
| Study | 17 | 89 | 4.6 | 77.5 |

… 3 more rows (all rows in the screenshot and the app)

## Q12 · Best-rated genres

*Which primary genres get the best star ratings, counting only genres with at least 5 ratings?*

INNER JOIN (4 tables) · GROUP BY · HAVING · COUNT · AVG · MIN · MAX

```
SELECT g.name                    AS genre,
       COUNT(*)                  AS ratings,
       ROUND(AVG(r.rating), 2)   AS avg_rating,
       MIN(r.rating)             AS lowest,
       MAX(r.rating)             AS highest
FROM ratings r
INNER JOIN songs       s  ON s.song_id   = r.song_id
INNER JOIN song_genres sg ON sg.song_id  = s.song_id AND sg.is_primary = 1
INNER JOIN genres      g  ON g.genre_id  = sg.genre_id
GROUP BY g.genre_id, g.name
HAVING COUNT(*) >= 5
ORDER BY avg_rating DESC, ratings DESC;
```

Output: 10 rows in 1.1 ms · screenshot Q12_best_rated_genres.png

| genre | ratings | avg_rating | lowest | highest |
|---|---|---|---|---|
| Folk | 11 | 4.09 | 2 | 5 |
| Film Dance | 14 | 3.86 | 2 | 5 |
| Ambient | 9 | 3.56 | 2 | 5 |
| Indie Pop | 27 | 3.52 | 2 | 5 |
| Lo-fi | 16 | 3.31 | 1 | 5 |
| EDM | 8 | 3.25 | 1 | 5 |
| Film Melody | 13 | 3.15 | 1 | 5 |
| Indie | 7 | 3.14 | 2 | 5 |

… 2 more rows (all rows in the screenshot and the app)

## Q13 · Listening by hour of day

*At what time of day does listening happen? Hours with no plays are shown as 0.*

Recursive CTE (hours 0-23) · LEFT JOIN · GROUP BY · COUNT · SUM · AVG · ORDER BY

```
WITH RECURSIVE hours AS (
  SELECT 0 AS h
  UNION ALL
  SELECT h + 1 FROM hours WHERE h < 23
)
SELECT hours.h                                      AS hour_of_day,
       COUNT(lh.play_id)                            AS plays,
       ROUND(COALESCE(SUM(lh.listened_sec), 0) / 3600, 1) AS hours,
       ROUND(AVG(lh.completion_pct), 1)             AS avg_completion_pct
FROM hours
LEFT JOIN listening_history lh ON HOUR(lh.started_at) = hours.h
GROUP BY hours.h
ORDER BY hour_of_day;
```

Output: 24 rows in 2.0 ms · screenshot Q13_listening_by_hour_of_day.png

| hour_of_day | plays | hours | avg_completion_pct |
|---|---|---|---|
| 0 | 44 | 2.2 | 82.1 |
| 1 | 56 | 2.7 | 77.7 |
| 2 | 0 | 0 | NULL |
| 3 | 0 | 0 | NULL |
| 4 | 0 | 0 | NULL |
| 5 | 62 | 4.2 | 82.2 |
| 6 | 189 | 10.5 | 84.1 |
| 7 | 336 | 17.8 | 80.2 |

… 16 more rows (all rows in the screenshot and the app)

## Q14 · Playlists with their real length

*What does each playlist contain, and how long is it really (computed from its songs)?*

LEFT JOIN (4 tables) · GROUP BY · COUNT · SUM · ORDER BY

```
SELECT pl.name                          AS playlist,
       p.display_name                   AS owner,
       COUNT(ps.song_id)                AS songs,
       ROUND(SUM(s.duration_sec) / 60, 1)  AS minutes,
       GROUP_CONCAT(s.title ORDER BY ps.position SEPARATOR ', ') AS songs_in_order
FROM playlists pl
INNER JOIN user_profiles p ON p.user_id = pl.user_id
LEFT JOIN playlist_songs ps ON ps.playlist_id = pl.playlist_id
LEFT JOIN songs s           ON s.song_id = ps.song_id
GROUP BY pl.playlist_id, pl.name, p.display_name
ORDER BY minutes DESC;
```

Output: 8 rows in 0.9 ms · screenshot Q14_playlists_with_their_real_length.png

| playlist | owner | songs | minutes | songs_in_order |
|---|---|---|---|---|
| Houseboat at 2AM | Leo Fernandes | 8 | 37 | Ennela Raatri, Chrome Horizon, Houseboat Blues, Kadal Alai, … |
| Rainy Tram | Ishita Sen | 8 | 33.6 | Godari Gaali, Pind Diyan Yaadan, Old Madras Radio, Safar, Tra… |
| Hyderabad Nights | Aarav Varma | 9 | 33 | Nachdi Raat, Barsaat Express, Gully Ka Shor, Gelupu Gelupu, … |
| Sangeet Night | Sana Qureshi | 7 | 26.8 | Nachdi Raat, Dhol Di Awaaz, Gelupu Gelupu, Neon Mela, Dha… |
| Leg Day Arsenal | Aarav Varma | 7 | 26.6 | Alai Payum Nenjam, Nandi Hills Run, Lohri Fire, Heart Rate Zer… |
| Sunday Morning Telugu | Priya Naidu | 6 | 25.4 | Kotha Kotha Ga, Nilave Nilave, Bheegi Khidki, Hosa Belaku, C… |
| Old Letters | Rohan Kulkarni | 5 | 20.9 | Ishq Ka Raasta, Vennela Vaana, Kaathal Mazhai, Paus Aala, C… |
| Deep Work Fuel | Meera Iyer | 5 | 16.8 | Chaayakkada, Local Train Window, Nocturne for an Empty Roo… |

## Q15 · Listeners above the average

*Which listeners have listened for longer than the average listener?*

GROUP BY | HAVING with a subquery | SUM | AVG

```
SELECT u.username,
       ROUND(SUM(lh.listened_sec) / 3600, 1) AS hours
FROM listening_history lh
INNER JOIN users u ON u.user_id = lh.user_id
GROUP BY u.user_id, u.username
HAVING SUM(lh.listened_sec) > (SELECT AVG(total_sec)
                               FROM (SELECT SUM(listened_sec) AS total_sec
                                     FROM listening_history
                                     GROUP BY user_id) per_user)
ORDER BY hours DESC;
```

Output: 7 rows in 5.0 ms · screenshot Q15_listeners_above_the_average.png

| username | hours |
|---|---|
| rohan | 15.3 |
| meera | 15.2 |
| aarav | 14.2 |
| ishita | 13.8 |
| priya | 13.7 |
| arjun | 13.5 |
| sana | 13.1 |

## Q16 · Each listener's favourite mood

*Which mood does each listener play most, and what share of their mood-tagged plays is it?*

INNER JOIN | GROUP BY | window function ROW_NUMBER() OVER (PARTITION BY) | CTE

```
WITH plays_by_mood AS (
  SELECT lh.user_id, m.name AS mood, COUNT(*) AS plays
  FROM listening_history lh
  INNER JOIN song_moods sm           ON sm.song_id = lh.song_id
  INNER JOIN intelligence_models im  ON im.model_id = sm.model_id AND im.is_active = 1
  INNER JOIN moods m                 ON m.mood_id = sm.mood_id
  GROUP BY lh.user_id, m.name
), ranked AS (
  SELECT user_id, mood, plays,
         ROUND(100 * plays / SUM(plays) OVER (PARTITION BY user_id), 1) AS share_pct,
         ROW_NUMBER() OVER (PARTITION BY user_id ORDER BY plays DESC, mood) AS rn
  FROM plays_by_mood
)
SELECT p.display_name AS listener, r.mood AS favourite_mood, r.plays, r.share_pct
FROM ranked r
INNER JOIN user_profiles p ON p.user_id = r.user_id
WHERE r.rn = 1
ORDER BY r.plays DESC;
```

Output: 11 rows in 12.4 ms · screenshot Q16_each_listeners_favourite_mood.png

| listener | favourite_mood | plays | share_pct |
|---|---|---|---|
| Aarav Varma | Motivational | 257 | 22.6 |
| Rohan Kulkarni | Nostalgic | 247 | 22.9 |
| Meera Iyer | Calm | 239 | 21.7 |
| Sana Qureshi | Motivational | 238 | 22.9 |
| Dev Sandhu | Motivational | 195 | 22 |
| Leo Fernandes | Focused | 176 | 19 |
| Ishita Sen | Nostalgic | 175 | 17.5 |
| Priya Naidu | Nostalgic | 153 | 14.6 |

… 3 more rows (all rows in the screenshot and the app)

## Q17 · Weekly MUSE Worlds and their recommendations

*For each weekly MUSE World, who is it for, what is its main mood, and how strong are its recommended songs?*

INNER JOIN (5 tables) GROUP BY COUNT AVG MAX

```
SELECT w.title                          AS world,
       p.display_name                   AS listener,
       m.name                           AS main_mood,
       w.week_start,
       COUNT(ri.song_id)                AS songs_recommended,
       ROUND(AVG(ri.final_score), 3)    AS avg_score,
       ROUND(MAX(ri.final_score), 3)    AS best_score
FROM weekly_worlds w
INNER JOIN user_profiles p         ON p.user_id = w.user_id
INNER JOIN moods m                 ON m.mood_id = w.primary_mood_id
INNER JOIN recommendation_batches b ON b.batch_id = w.batch_id
INNER JOIN recommendation_items ri ON ri.batch_id = b.batch_id
GROUP BY w.world_id, w.title, p.display_name, m.name, w.week_start
ORDER BY avg_score DESC;
```

Output: 11 rows in 1.3 ms · screenshot Q17_weekly_muse_worlds_and_their_recommendations.png

| world | listener | main_mood | week_start | songs_recommended | avg_score | best_score |
|---|---|---|---|---|---|---|
| Firework Courtyard | Arjun Shetty | Euphoric | 2026-10-26 | 18 | 0.653 | 0.782 |
| Lantern Courtyard | Priya Naidu | Euphoric | 2026-10-26 | 18 | 0.64 | 0.737 |
| Sepia Matinee | Rohan Kulkarni | Nostalgic | 2026-10-26 | 18 | 0.627 | 0.725 |
| Lantern Procession | Aarav Varma | Euphoric | 2026-10-26 | 18 | 0.624 | 0.758 |
| Firework Courtyard | Dev Sandhu | Euphoric | 2026-10-26 | 18 | 0.62 | 0.747 |
| Sepia Afternoons | Ishita Sen | Nostalgic | 2026-10-26 | 18 | 0.608 | 0.701 |
| Thousand-Lamp Celebration | Sana Qureshi | Euphoric | 2026-10-26 | 18 | 0.603 | 0.739 |
| First-Light Path | Kiran Das | Peaceful | 2026-10-26 | 18 | 0.578 | 0.723 |

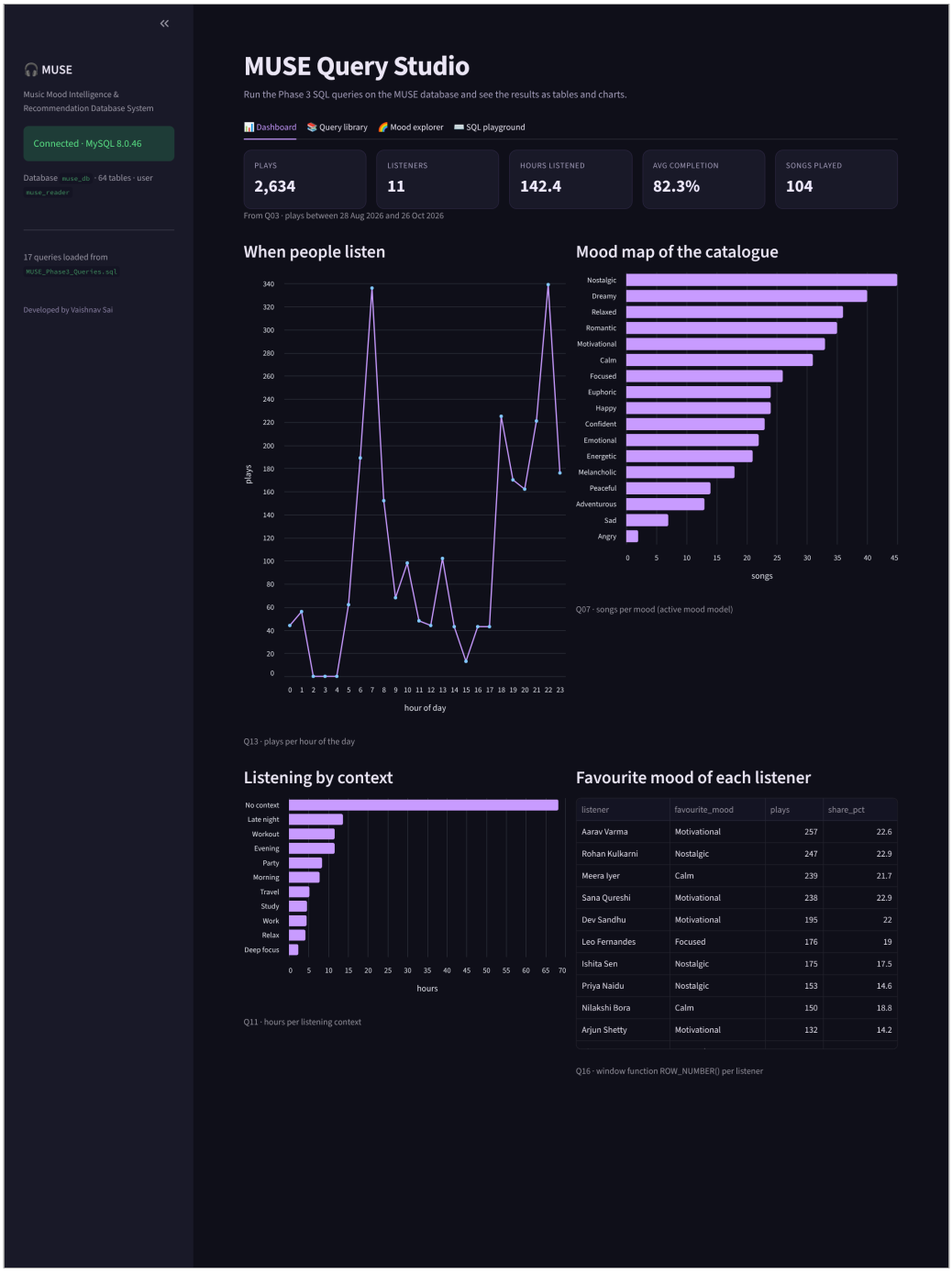
… 3 more rows (all rows in the screenshot and the app)

## 5. Interface — MUSE Query Studio

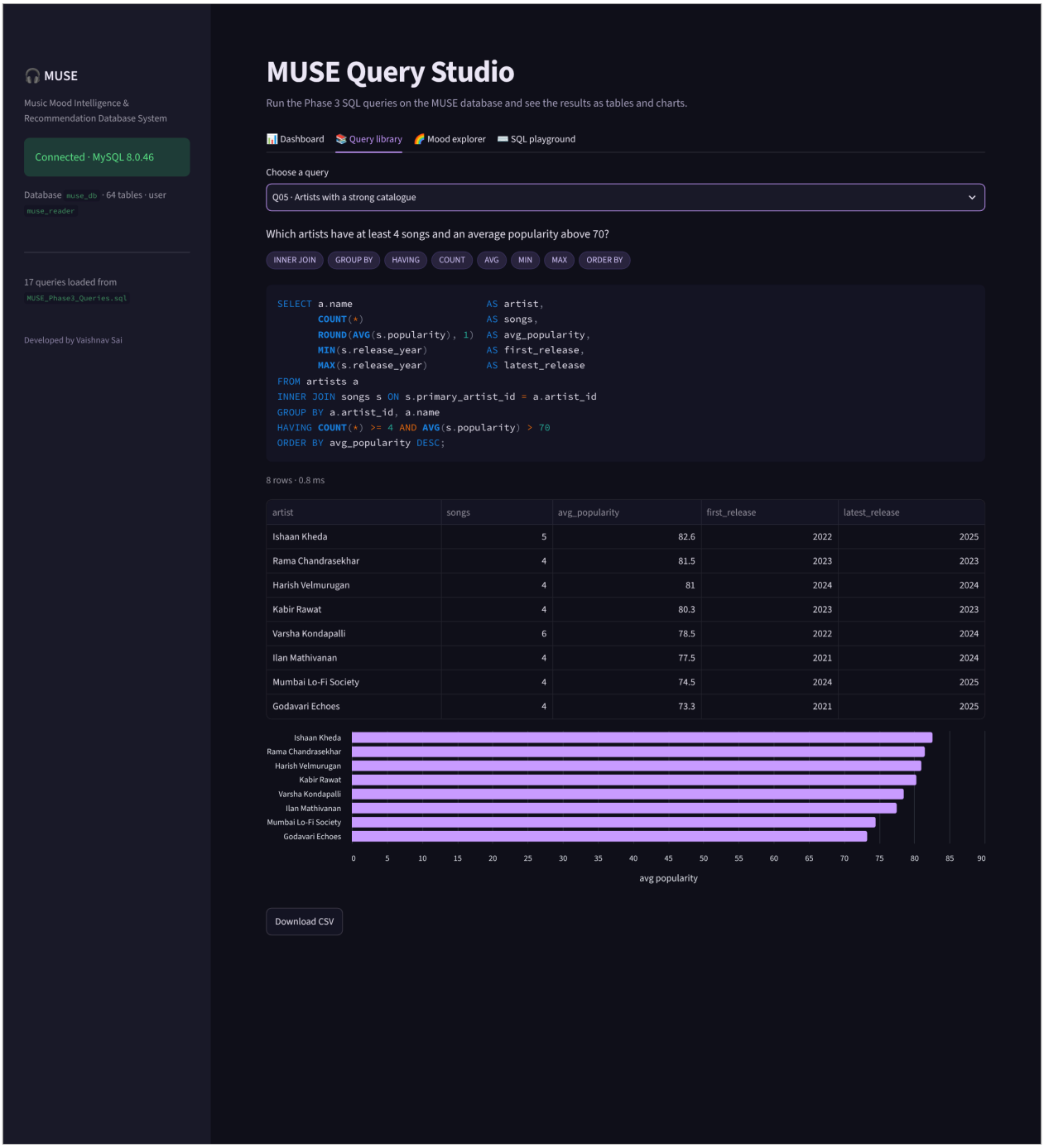
A Streamlit web app (`phase3/app/streamlit_app.py`) connects to `muse_db` and has four tabs:

- **Dashboard** — headline figures (Q03) and charts for listening by hour (Q13), the mood map (Q07), listening by context (Q11) and each listener's favourite mood (Q16).
- **Query library** — pick any of the 17 queries: the question, the concepts it uses, the SQL, and the result as a table and (where useful) a chart, with row count, run time and a CSV download.
- **Mood explorer** — choose a mood, a minimum confidence and a language; a *parameterised* query (values sent separately from the SQL) lists matching songs and plots them by energy and positivity.
- **SQL playground** — type your own SELECT. Only one SELECT/WITH statement is accepted, and it runs inside `START TRANSACTION READ ONLY`; the recommended account `muse_reader` also has SELECT privilege only.

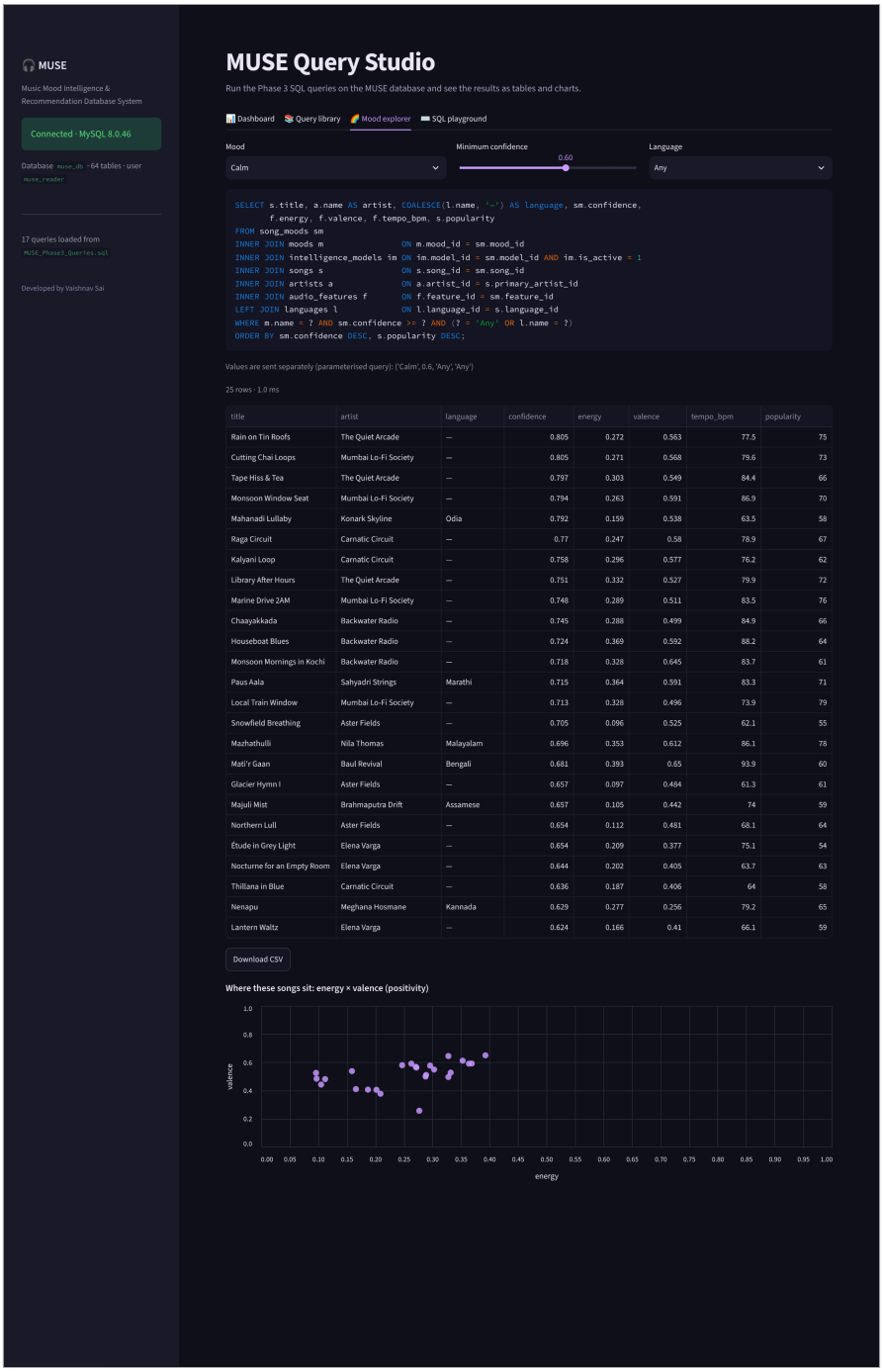
**Run it:** load `database/phase2/schema.sql` in Workbench, then in `phase3/app`: `pip install -r requirements.txt`, copy `.env.example` to `.env` and enter your MySQL user and password (optionally create the read-only user with `create_reader_user.sql`), and run `streamlit run streamlit_app.py`. The same queries run in a terminal with `python muse_cli.py run Q05` (or `list`, `run all`).



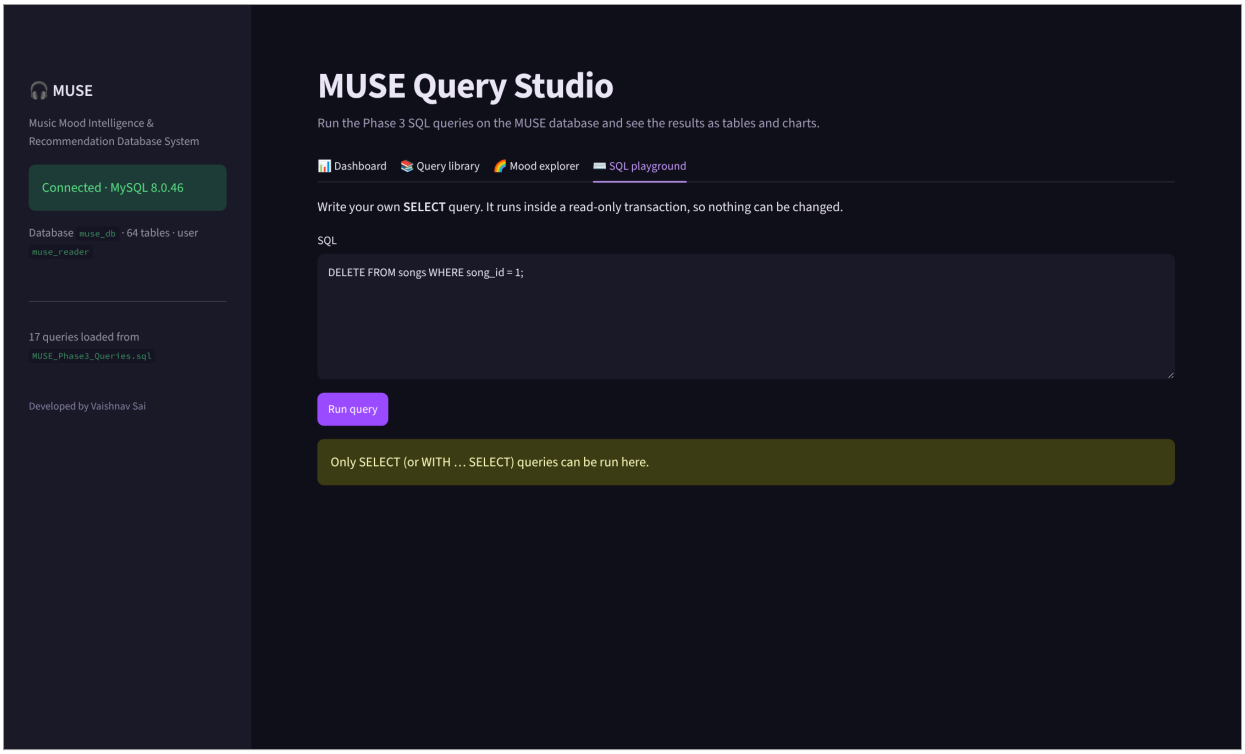

Dashboard tab (real screenshot of the running app)




Query library: Q05 with its SQL, output table and chart




Mood explorer: parameterised query for the mood Calm




SQL playground refusing a DELETE statement

## 6. Testing

Each query was run on MySQL 8.0.46-0ubuntu0.24.04.4 as user `muse_reader` (read-only) by `phase3/run_query_tests.py`, which uses the same code as the app. A query passes if it runs without error and returns at least one row. The whole query file was also executed in the MySQL command-line client without errors, and every screenshot was taken from the running app. Result: **17 of 17 queries pass**.

| ID | Query | Rows | ms | Result |
|---|---|---|---|---|
| Q01 | Popular recent songs | 15 | 0.9 | Pass |
| Q02 | Calm-sounding tracks by audio profile | 19 | 0.9 | Pass |
| Q03 | Platform listening summary | 1 | 2.1 | Pass |
| Q04 | Listener leaderboard | 11 | 5.7 | Pass |
| Q05 | Artists with a strong catalogue | 8 | 0.8 | Pass |
| Q06 | Most-skipped songs | 10 | 5.3 | Pass |
| Q07 | Mood map of the catalogue | 17 | 1.9 | Pass |
| Q08 | Every mood with its song count | 17 | 1.2 | Pass |
| Q09 | Songs no one has put in a playlist | 28 | 0.8 | Pass |
| Q10 | Catalogue by language | 12 | 0.8 | Pass |
| Q11 | Listening by context | 11 | 4.9 | Pass |
| Q12 | Best-rated genres | 10 | 1.1 | Pass |
| Q13 | Listening by hour of day | 24 | 2.0 | Pass |
| Q14 | Playlists with their real length | 8 | 0.9 | Pass |
| Q15 | Listeners above the average | 7 | 5.0 | Pass |
| Q16 | Each listener's favourite mood | 11 | 12.4 | Pass |
| Q17 | Weekly MUSE Worlds and their recommendations | 11 | 1.3 | Pass |

### Read-only guard

| Statement tried in the interface | Result |
|---|---|
| `DELETE FROM songs WHERE song_id = 1` | Rejected — Only SELECT (or WITH … SELECT) queries can be run here. |
| `SELECT 1; DROP TABLE songs` | Rejected — Run one statement at a time. |
| `UPDATE songs SET popularity = 0` | Rejected — Only SELECT (or WITH … SELECT) queries can be run here. |
| `SELECT * FROM songs INTO OUTFILE '/tmp/x.csv'` | Rejected — This kind of SELECT is not allowed here (it writes files, locks rows or delays the server). |

These tests used the MySQL command-line client and the app. MySQL Workbench 8.0 CE sends the same statements to the same MySQL 8.0 server, and the file uses only standard MySQL 8.0 syntax.

## 7. Conclusion

The MUSE database answers questions a real music service would ask — what people listen to and when, which songs they skip, how the catalogue spreads across moods and languages, and how strong the weekly recommendations are — using the SELECT, WHERE, aggregate, GROUP BY / HAVING / ORDER BY and INNER / LEFT / RIGHT JOIN queries required by the brief, plus a subquery, CTEs and a window function. The Streamlit interface makes the same queries usable without writing SQL, while keeping the database read-only.

Developed by Vaishnav Sai.